🐳 LLM + Container

Lab Worksheet — Ollama, Typhoon & Gradio บน HPC

RSU HPC AI Data Center • 2026

ชื่อ - นามสกุล		วันที่

Username (HPC)	

🎯 วัตถุประสงค์การเรียนรู้

- รัน Ollama บน Docker (CPU mode) บน rsu-training และทดสอบ REST API ได้
- ส่ง SLURM job เพื่อรัน Ollama + Gradio UI บน MIG GPU พร้อมกันใน script เดียว
- อ่าน port_forwarding_command ที่ generate อัตโนมัติ และ SSH tunnel เพื่อเข้า Gradio UI
- Pull model Typhoon2-8b ผ่าน Gradio UI และ chat เป็นภาษาไทยได้
- เข้าใจพารามิเตอร์ temperature และผลกระทบต่อ output ของ LLM
- เข้าใจว่า conda env และ Ollama เตรียมไว้พร้อมแล้ว — ไม่ต้อง install เพิ่ม

ส่วนที่ 1 — พื้นฐาน: Docker, Container & LLM

1.1 Ollama คืออะไร?

Ollama คือ tool ที่ช่วย run LLM บน local machine ได้ง่ายๆ รองรับทั้ง Mac, Linux, และ Windows รวมถึงมี REST API สำหรับ integration

🐳 Docker mode (CPU)	⚡ Ollama userspace (GPU)	🌐 Gradio UI
ง่าย เร็ว ใช้งาน demo concept ได้ แต่ run บน CPU เท่านั้น ต้องใช้ --runtime=runc	ติดตั้งไว้ใน userspace แล้ว รัน ollama serve โดยตรงผ่าน SLURM job ได้ GPU จริง	Web UI ของ ollama_web.py รัน port random — เข้าถึงได้ผ่าน SSH tunnel

1.2 Dynamic Port — ไม่ชนกันระหว่าง user

เนื่องจากมีหลาย user ใช้งาน rsu-training พร้อมกัน port ของ Ollama และ Gradio จึงต้องไม่ซ้ำกัน script จะเลือก port ว่างอัตโนมัติด้วยฟังก์ชัน get_free_port():

```
# ฟังก์ชันใน ollama_gradio_run.sh
get_free_port() {
  while true; do
    PORT=$(shuf -i 20000-60000 -n 1)   # สุ่ม port 20000-60000
    if ! ss -ltn | grep -q ":$PORT$"; then
      echo $PORT  # port ว่าง → ใช้ได้
      return
    fi
  done
}

PORT_GRADIO=$(get_free_port)   # เช่น 34521
OLLAMA_PORT=$(get_free_port)   # เช่น 47832

# SSH command ถูก save ไว้ในไฟล์อัตโนมัติ
echo "ssh -L ${PORT_GRADIO}:$(hostname):${PORT_GRADIO} ..." >
port_forwarding_command
```

ทำไมต้องใช้ port random แทนที่จะกำหนด port ตายตัวเช่น 7860?

ส่วนที่ 2 — Lab A: Docker Ollama (CPU mode)

A **Docker Ollama บน rsu-training**
ใช้ CPU เท่านั้น (--runtime=runc) | model: tinyllama

โปรดทำตามลำดับขั้นตอน ถ้าติดปัญหาให้ดูที่ Troubleshooting Box

Step 1 — คำนวณ port และ run container

```
# คำนวณ port ของตัวเอง
MY_PORT=$((11400 + $(id -u) % 1000))
echo "My Ollama port: $MY_PORT"

# Run Ollama container (ไม่ใช้ GPU — CPU mode)
docker run -d --runtime=runc \
  --name ollama-$USER \
  -p ${MY_PORT}:11434 \
  -v $HOME/ollama_models:/root/.ollama \
  ollama/ollama

# ตรวจสอบ container
docker ps
```

Port ที่ได้คือ? Container ID (12 ตัวแรก) คือ?

Step 2 — Pull model และ chat

```
# Pull tinyllama (ขนาด ~637 MB)
docker exec -it ollama-$USER ollama pull tinyllama

# Chat ใน terminal
docker exec -it ollama-$USER \
  ollama run tinyllama "What is a container?"

# ดู model ที่ download ไว้
curl http://localhost:${MY_PORT}/api/tags
```

*คำตอบที่ได้จาก **tinyllama** เกี่ยวกับ **'What is a container?'** (สรุปสั้น ๆ)*

Step 3 — REST API test

```
# ส่ง request ผ่าน REST API
curl http://localhost:${MY_PORT}/api/chat -d '{
  "model": "tinyllama",
  "messages": [{"role": "user", "content": "Hello! What can you do?"}],
  "stream": false
}'

# สังเกต field: message.content, total_duration, eval_count
```

*จาก **JSON response: total_duration** และ **eval_count** คืออะไร? หมายความว่าอย่างไร?*

🔧 **Troubleshooting — Docker**

- docker: Error response: error while creating mount sources → ลอง restart container: docker rm -f ollama-$USER แล้วรัน docker run ใหม่
- port is already in use → มี container เดิมอยู่ ใช้ docker ps แล้ว docker rm -f <name>
- connection refused → container ยัง start ไม่เสร็จ รอ 5-10 วินาทีแล้วลองใหม่

ส่วนที่ 3 — Lab B: SLURM + Ollama + Typhoon (GPU จริง)

B

SLURM + Ollama + Gradio บน MIG GPU

Ollama ติดตั้งไว้แล้ว — แค่ sbatch แล้วเปิด browser | model: Typhoon2-8b

✅ **สิ่งที่เตรียมไว้พร้อมแล้ว — ไม่ต้อง install เพิ่ม**

- Ollama binary: ติดตั้งใน userspace แล้ว (~/ollama/bin/ollama หรือ PATH)
- conda env 'deepseek': มี gradio, requests, torch ครบแล้ว
- repo: ~/deepseek-with-ollama-on-supercomputer/ มีไฟล์ ollama_gradio_run.sh และ ollama_web.py
- สิ่งที่ผู้เรียนต้องทำ: cd repo → sbatch → cat port_forwarding_command → SSH → browser

Step 1 — เข้า repo และตรวจสอบ environment

```
# เข้า repo ที่เตรียมไว้
cd ~/deepseek-with-ollama-on-supercomputer
ls -la

# Activate conda environment
conda activate deepseek

# ตรวจสอบว่า ollama ใช้งานได้
ollama --version

# ดูไฟล์ SLURM script
cat ollama_gradio_run.sh
```

ollama version ที่ได้คืออะไร? ไฟล์ใน repo มีอะไรบ้าง?

Step 2 — Submit SLURM Job

script จะ: (1) เลือก port ว่างอัตโนมัติ (2) รัน Ollama serve บน GPU (3) รอ Ollama พร้อม (4) รัน Gradio UI พร้อมกัน

```
# Submit job
sbatch ollama_gradio_run.sh

# ดู status (R = Running, PD = Pending)
squeue -u $USER

# ดู log real-time (รอ ~30-60 วินาที)
tail -f slurm-<JOBID>.out
```

```
# สังเกต output ที่บอกว่าพร้อม:
# ✅ Ollama server is ready!
# ✅ Gradio is running!
# 🎉 All services started successfully!
```

JOBID ที่ได้คือ? job อยู่บน node ไหน (NODELIST)?

Step 3 — อ่าน SSH Command ที่ Generate ให้อัตโนมัติ

💡 **ทำไม SSH Tunnel ถึงจำเป็น?**

- Gradio UI รันอยู่บน rsu-training (compute node) ซึ่งไม่ได้ expose ออก internet โดยตรง
- SSH Tunnel สร้าง 'ท่อ' จาก localhost ของเรา ผ่าน rsu-login ไปถึง rsu-training
- Script บันทึก command ที่ถูกต้องไว้ในไฟล์ port_forwarding_command ให้เลย ไม่ต้องพิมพ์เอง

```
# อ่าน SSH command ที่ generate อัตโนมัติ
cat port_forwarding_command

# ตัวอย่าง output ที่จะเห็น:
# ssh -L localhost:34521:rsu-training:34521 test06@171.102.216.177

# เปิด Terminal ใหม่บนเครื่องของตัวเอง (ไม่ใช่บน HPC)
# แล้ว copy-paste command นั้นทั้งหมด
# → ใส่ password แล้วปล่อย Terminal นั้นไว้ (อย่าปิด)
```

SSH command ที่ได้จาก cat port_forwarding_command คือ? (copy มาเขียน)

Step 4 — เปิด Gradio UI และ Pull Typhoon Model

หลังจาก SSH tunnel เปิดอยู่แล้ว เปิด browser แล้วเข้า URL ต่อไปนี้ (ใช้ port เดียวกับใน SSH command)

```
# Browser URL (port ตรงกับ SSH tunnel):
http://localhost:<PORT>

# ใน Gradio UI:
# 1. ดูที่ช่อง 'Model Name to Pull' ด้านขวา
# 2. พิมพ์: scb10x/llama3.1-typhoon2-8b-instruct
# 3. คลิก 'Pull Model'
# 4. รอ download ~5 GB (ครั้งแรก — ครั้งถัดไปใช้ได้ทันที)
# 5. เมื่อ Pull สำเร็จ model จะปรากฏใน dropdown 'Select Model'
```

URL ที่ใช้เปิด Gradio คือ? ขณะ pull model เห็น Pull Status ว่าอะไร?

Step 5 — Chat กับ Typhoon เป็นภาษาไทย

เลือก model scb10x/llama3.1-typhoon2-8b-instruct ใน dropdown แล้วทดสอบ chat

```
# ใน Gradio UI — ลองส่ง message เหล่านี้:

สวัสดีครับ คุณคือใคร?

ช่วยสรุป LLM คืออะไร ใน 3 ประโยค

เขียน Python code สำหรับ fibonacci 10 ตัวแรก

# สังเกต: temperature slider ด้านขวา
# ลองเปลี่ยน temperature แล้วถามคำเดิมซ้ำ
```

***Typhoon** แนะนำตัวเองว่าอะไร? ตอบได้ลื่นแค่ไหน?*

🔧 **Troubleshooting — Lab B**

- squeue ไม่เห็น job: รอ 5-10 วินาที SLURM อาจช้าช่วง peak hours
- tail -f log แต่ไม่เห็น ✅ Ollama ready: รอนานขึ้น GPU อาจยุ่งอยู่ — ปกติไม่เกิน 2 นาที
- SSH tunnel error 'bind: address already in use': port นั้น tunnel อยู่แล้ว ลอง ssh -L localhost:PORT2:... ด้วย port อื่น
- Browser เปิดแล้วเห็น 'Connection refused': SSH tunnel ยังไม่ได้รัน หรือ port ผิด
- Pull ค้าง นานมาก: ปกติ ~5 GB ใช้เวลา 5-15 นาที — ถ้า error ให้ลอง Pull ใหม่
- จบงาน: scancel <JOBID> เพื่อ free GPU — model ที่ download ไว้ใน ~/ollama/models จะยังอยู่

ส่วนที่ 4 — Lab C: Temperature Parameter

C ทดลอง **Temperature** บน **Gradio UI**
เปรียบเทียบผลที่ได้ที่ temperature ต่างกัน | model: Typhoon2-8b

Temperature กำหนดความ 'สุ่ม' ของการตอบ: ค่าต่ำ = แน่นอน, ค่าสูง = หลากหลาย/สร้างสรรค์

ช่วง Temperature	ลักษณะ	เหมาะกับงาน
0.0 – 0.3	Deterministic	Code generation, factual Q&A, คณิตศาสตร์ — ตอบเหมือนเดิมทุกครั้ง
0.5 – 0.8	Balanced	Chatbot ทั่วไป, อธิบาย concept — มีความหลากหลายเล็กน้อย
1.0 – 2.0	Creative	เขียน story, brainstorm, creative writing — ต่างกันทุกครั้ง

การทดลอง: Creative vs Deterministic

ใช้ prompt เดิมซ้ำ 3 ครั้งต่อ temperature — สังเกตว่าเปลี่ยนหรือไม่

📝 **Prompt ที่แนะนำ (ทดสอบแล้วได้ผลชัดเจน)**

แมวตัวหนึ่งเดินเข้าร้านกาแฟ แล้ว

→ ไม่มีคำตอบที่ 'ถูก' — model ต่อเรื่องได้อิสระ ทำให้ temperature มีผลชัดเจนที่สุด

Temperature	ผลลัพธ์ (รอบ 1, 2, 3 เหมือนหรือต่างกัน?)	สังเกตุ: สรุปลักษณะของคำตอบ
0.0 *ควรเหมือนกันทุกครั้ง*		
0.8 *ควรแตกต่างกันบ้าง*		
1.2 *ควรต่างกันมาก ทุกรอบ*		

*จากผลลัพธ์ที่ได้: **temperature 0.0** กับ **1.2** ต่างกันอย่างไร? และควรใช้ค่าไหนกับงานประเภทใด?*

ส่วนที่ 5 — Checklist สรุป

ทำเครื่องหมาย ☑ ในช่องที่ทำสำเร็จแล้ว

☐	คำนวณ dynamic port ของตัวเองได้
☐	Run Docker Ollama container ด้วย --runtime=runc สำเร็จ
☐	Pull tinyllama และ chat ใน terminal ได้
☐	ทดสอบ REST API /api/chat ด้วย curl
☐	cd repo และ conda activate deepseek สำเร็จ
☐	ollama --version ใช้งานได้ (userspace install)
☐	Submit SLURM job ด้วย sbatch และดู log ด้วย tail -f
☐	อ่าน port_forwarding_command และ copy SSH command
☐	สร้าง SSH Tunnel สำเร็จและเปิด Gradio UI จาก browser
☐	Pull scb10x/llama3.1-typhoon2-8b-instruct สำเร็จ
☐	Chat กับ Typhoon เป็นภาษาไทยได้
☐	ทดลอง temperature ต่างๆ และจด output
☐	scancel <JOBID> เพื่อ free GPU เมื่อเสร็จงาน

📝 คำถาม **Reflection**

1. ความแตกต่างระหว่าง Docker mode (CPU) กับ Ollama userspace + SLURM (GPU) คืออะไร? ควรใช้แบบไหนในสถานการณ์ไหน?

2. ทำไม script ถึงเลือก port random (get_free_port) แทนที่จะกำหนดค่าตายตัว? ถ้าต้องการให้ผู้เรียน 20 คนใช้พร้อมกันจะเกิดอะไรขึ้นถ้า port ตายตัว?

3. temperature = 0.1 กับ temperature = 1.0 ให้ผลต่างกันอย่างไร? ควรใช้ค่าไหนสำหรับ code generation? และสำหรับ creative writing?